

Relatório Trabalho Prático Álgebra Abstrata

Luma Guimaraes

1

1. Introdução

Este relatório apresenta a implementação de três algoritmos fundamentais da Teoria dos Números e da Criptografia: o teste de primalidade de Miller–Rabin, o criptosistema RSA (Rivest–Shamir–Adleman) e o algoritmo de Tonelli–Shanks para cálculo de raízes quadradas modulares integrados.[1]

2. Descrição da Implementação

A implementação foi desenvolvida em Python e organizada em funções independentes, permitindo o reaproveitamento de código entre os diferentes algoritmos.

Inicialmente foi implementada uma função de exponenciação modular rápida (`mod_pow`), utilizada em praticamente todas as etapas do projeto. Essa função é responsável por calcular potências módulo um inteiro de forma eficiente, evitando o crescimento excessivo dos valores intermediários.

Também foram implementados os algoritmos de Euclides e Euclides Estendido. O primeiro é utilizado para verificar se dois números são coprimos, enquanto o segundo é empregado no cálculo de inversos modulares, necessários para a geração das chaves RSA.

A implementação do Miller–Rabin foi dividida em duas partes. A primeira realiza uma rodada individual do teste utilizando uma base aleatória. A segunda executa múltiplas rodadas e decide se o número deve ser considerado composto ou provavelmente primo. Para gerar números primos, o programa cria candidatos ímpares aleatórios com tamanho fixo em bits e repete o teste até encontrar um valor aprovado por todas as rodadas.

O algoritmo de Tonelli–Shanks foi implementado utilizando o Critério de Euler para verificar previamente se existe solução para a congruência. Quando o valor analisado é um resíduo quadrático, o algoritmo executa as etapas necessárias para encontrar uma raiz quadrada modular. Também foram adicionadas verificações para confirmar que as raízes encontradas realmente satisfazem à congruência original.

A implementação do RSA aproveita diretamente os números primos gerados pelo módulo de Miller–Rabin. Após a obtenção dos primos (p) e (q), são calculados o módulo (n), a função totiente e os expoentes público e privado. O Euclides Estendido é utilizado para determinar o inverso modular correspondente à chave privada.

As operações de cifração, decifração, assinatura digital e verificação de assinaturas foram implementadas utilizando a mesma função de exponenciação modular desenvolvida no início do projeto. Toda essa descrição reforça o ponto forte da implementação, que é as subrotinas de funções matemáticas. Ela reduz a quantidade de código duplicado e facilitou os testes, uma vez que a corretude da exponenciação modular e do Euclides Estendido beneficia simultaneamente os módulos de Miller–Rabin, Tonelli–Shanks e RSA.

3. Miller-Rabin

3.1. Teste de primalidade de Miller-Rabin

O teste de primalidade de Miller-Rabin usa como base o teste de primalidade de Fermat (se p é primo e $p \nmid a$, então $a^{p-1} \equiv 1 \pmod{p}$)[2]. Porém, note que não é uma implicação dupla: existem números compostos que satisfazem essa congruência para determinadas bases a , conhecidos como **pseudoprimos**. Portanto, o teste de Fermat isoladamente não é suficiente para garantir a primalidade de um número.

O algoritmo de Miller-Rabin fortalece essa verificação explorando propriedades adicionais dos números primos. Seja $n > 2$ um inteiro ímpar candidato à primalidade. Inicialmente, escreve-se

$$n - 1 = 2^s \cdot d,$$

onde d é ímpar. Em seguida, escolhe-se aleatoriamente uma base a tal que $2 \leq a \leq n - 2$ e calcula-se

$$x = a^d \pmod{n}.$$

Se $x \equiv 1 \pmod{n}$ ou $x \equiv -1 \pmod{n}$, o número n passa nessa rodada do teste. Caso contrário, calcula-se sucessivamente

$$x^2, x^4, x^8, \dots, x^{2^{s-1}} \pmod{n}.$$

Se em alguma dessas etapas for obtido o valor $-1 \pmod{n}$, então n também passa na rodada. Caso nenhum dos valores calculados seja congruente a $-1 \pmod{n}$, conclui-se que n é composto.

A justificativa matemática do teste baseia-se no fato de que, para um número primo p , as únicas raízes quadradas de 1 módulo p são 1 e -1 . Assim, ao elevar sucessivamente os valores obtidos, qualquer desvio desse comportamento fornece um certificado de que o número analisado é composto.

3.2. Limitações

Como o algoritmo é probabilístico, uma única execução não garante que um número seja primo. Entretanto, cada rodada reduz significativamente a probabilidade de erro. Mais especificamente, para qualquer número composto, a probabilidade de ele ser incorretamente classificado como primo em uma rodada é inferior a $1/4$. Portanto, após k rodadas independentes, a probabilidade de erro é menor que [3]

$$\left(\frac{1}{4}\right)^k.$$

Na implementação deste trabalho, o teste de Miller-Rabin foi utilizado para verificar a primalidade dos candidatos gerados aleatoriamente durante a construção dos números primos empregados pelo criptossistema RSA.

3.3. Geração de Primos de k Bits

Inicialmente, foram gerados números ímpares aleatórios com comprimentos de 32, 64 e 128 bits. Cada candidato foi submetido ao teste de Miller–Rabin com 11 iterações independentes. Caso o candidato fosse considerado composto, um novo número era gerado até que um provável primo fosse encontrado.

Para cada tamanho de entrada, o procedimento foi executado 10 vezes, registrando-se a quantidade de candidatos testados até a obtenção de um número primo.

A média observada foi comparada com a estimativa teórica baseada no Teorema dos Números Primos. A probabilidade de um inteiro aleatório próximo de N ser primo é aproximadamente

$$\frac{1}{\ln N}.$$

Como apenas números ímpares são gerados, a quantidade esperada de tentativas até encontrar um primo é aproximadamente

$$\ln(2^k) \frac{1}{2}$$

$$k \ln 2 \frac{1}{2 \approx 0,347k}.$$

Os resultados experimentais mostraram boa concordância com essa previsão teórica, indicando que o gerador de candidatos e a implementação do teste de primalidade estão funcionando corretamente.

3.4. Validação em Casos Conhecidos

Além da geração de primos, o algoritmo foi testado utilizando números previamente conhecidos, tanto primos quanto compostos.

Foram utilizados os seguintes exemplos:

- 101 (primo);
- 1009 (primo);
- $91 = 7 \times 13$ (composto);
- 561 (número de Carmichael);
- $2047 = 23 \times 89$ (pseudoprimo para algumas bases).

Os números primos foram corretamente identificados como “provavelmente primos”, enquanto os compostos foram classificados como compostos. Em particular, os casos de Carmichael e pseudoprimos foram incluídos por serem exemplos clássicos que podem enganar testes probabilísticos mais simples, demonstrando a robustez do algoritmo de Miller–Rabin.

3.5. Conclusão final dos testes

Os experimentos confirmaram a corretude da implementação. A geração de primos apresentou comportamento compatível com a teoria analítica dos números, e os testes com exemplos conhecidos demonstraram a capacidade do algoritmo de distinguir eficientemente números primos de números compostos, inclusive em casos tradicionalmente difíceis para outros testes de primalidade.

4. Fundamentação Teórica do Criptossistema RSA

O RSA (Rivest–Shamir–Adleman) é um criptossistema de chave pública baseado na dificuldade computacional da fatoração de números inteiros grandes.

A construção do sistema inicia-se pela escolha de dois números primos distintos p e q . A partir deles calcula-se

$$n = pq$$

e a função totiente de Euler

$$\varphi(n) = (p - 1)(q - 1).$$

Em seguida, escolhe-se um inteiro e tal que

$$\gcd(e, \varphi(n)) = 1.$$

A chave privada é obtida calculando o inverso modular de e módulo $\varphi(n)$, ou seja, um valor d que satisfaz

$$ed \equiv 1 \pmod{\varphi(n)}.$$

A cifração de uma mensagem m é realizada por

$$c \equiv m^e \pmod{n},$$

enquanto a decifração utiliza

$$m \equiv c^d \pmod{n}.$$

A corretude do RSA decorre do Teorema de Euler, que afirma que, para inteiros coprimos com n ,

$$m^{\varphi(n)} \equiv 1 \pmod{n}.$$

Como os expoentes e e d são escolhidos de forma que $ed \equiv 1 \pmod{\varphi(n)}$, segue que a aplicação sucessiva da cifração e da decifração recupera a mensagem original.

Além de garantir confidencialidade, o RSA também permite a implementação de assinaturas digitais. Nesse caso, a operação realizada com a chave privada produz uma assinatura que pode ser validada por qualquer usuário que possua a chave pública correspondente.

A segurança do RSA depende da dificuldade prática de fatorar o número n para recuperar os primos p e q , problema para o qual não existe atualmente um algoritmo eficiente conhecido para computadores clássicos.

4.1. Testes e Resultados do Criptossistema RSA

Após a implementação dos algoritmos auxiliares, foi desenvolvido o criptossistema RSA completo, incluindo geração de chaves, cifração, decifração, assinatura digital e verificação de assinaturas.

4.2. Geração de Chaves

A geração das chaves segue as etapas clássicas do RSA:

geração de dois primos distintos p e q ; cálculo de $n = pq$;

cálculo da função totiente $\varphi(n) = (p - 1)(q - 1)$;

escolha de um expoente público e coprimo com $\varphi(n)$;

cálculo do inverso modular d de e módulo $\varphi(n)$.

Os primos p e q foram obtidos utilizando exclusivamente o gerador baseado no teste de Miller–Rabin desenvolvido neste trabalho.

4.3. Teste de Cifração e Decifração

Para validar a implementação, foram realizadas 20 execuções independentes.

Em cada execução:

uma mensagem aleatória m foi gerada; a mensagem foi cifrada:

$$c = m^e \bmod n;$$

o texto cifrado foi decifrado:

$$m' = c^d \bmod n;$$

verificou-se se $m' = m$.

Todos os testes retornaram sucesso, demonstrando a corretude do processo de cifração e decifração.

4.4. Teste de Assinatura Digital

Além da confidencialidade, o RSA também foi utilizado para autenticação por meio de assinaturas digitais.

Em cada execução:

a mensagem foi assinada:

$$s = m^d \bmod n;$$

a assinatura foi verificada:

$$m_v = s^e \bmod n;$$

verificou-se se $m_v = m$.

Todas as assinaturas produzidas foram verificadas com sucesso.

4.5. Conclusão final dos testes

Os resultados experimentais demonstraram que o criptossistema RSA foi implementado corretamente. Todas as operações de geração de chaves, cifração, decifração e assinatura digital produziram os resultados esperados, validando a integração entre os módulos de Miller–Rabin, Euclides Estendido, exponenciação modular e aritmética modular desenvolvidos neste trabalho.

5. Fundamentação Teórica do Algoritmo de Tonelli–Shanks

O algoritmo de Tonelli–Shanks resolve o problema de encontrar uma raiz quadrada modular, isto é, determinar um valor x tal que

$$x^2 \equiv a \pmod{p},$$

onde p é um número primo ímpar.

Antes de procurar a solução, é necessário verificar se ela existe. Para isso utiliza-se o Critério de Euler, que afirma que um inteiro a é um resíduo quadrático módulo p se, e somente se,

$$a^{\frac{p-1}{2}} \equiv 1 \pmod{p}.$$

Quando essa condição não é satisfeita, a congruência não possui solução.

O algoritmo baseia-se na decomposição de

$$p - 1 = 2^s q,$$

onde q é ímpar. Em seguida, escolhe-se um não-resíduo quadrático z módulo p e utiliza-se uma sequência de exponenciações modulares para reduzir iterativamente o problema até obter uma raiz quadrada de a .

A corretude do método decorre da estrutura multiplicativa do corpo finito $\mathbb{Z}_p$ e do fato de que a ordem dos elementos pode ser manipulada através da decomposição de $p - 1$ em uma potência de dois multiplicada por um fator ímpar.

O algoritmo é amplamente utilizado em criptografia e teoria dos números computacional por permitir o cálculo eficiente de raízes quadradas modulares mesmo para números primos de grande tamanho.

5.1. Testes e Resultados do Algoritmo de Tonelli–Shanks

O algoritmo de Tonelli–Shanks foi implementado para calcular raízes quadradas modulares da congruência

$$x^2 \equiv a \pmod{p},$$

onde p é um número primo ímpar e a é um resíduo quadrático módulo p .

5.2. Verificação da Solubilidade

Antes da execução do algoritmo, foi aplicada uma verificação baseada no Critério de Euler. Para um número primo p , um inteiro a possui raiz quadrada módulo p se, e somente se,

$$a^{\frac{p-1}{2}} \equiv 1 \pmod{p}.$$

Essa etapa foi implementada através do símbolo de Legendre e garante que o algoritmo seja executado apenas quando existe solução.

5.3. Exemplos Obrigatórios

Foram executados os exemplos especificados no enunciado do trabalho:

- $x^2 \equiv 5 \pmod{41}$;
- $x^2 \equiv 2 \pmod{113}$.

Para cada caso, o algoritmo retornou uma raiz r . A segunda raiz foi obtida pela propriedade

$p-r$.

As soluções encontradas foram verificadas substituindo-as diretamente na congruência original.

Além disso, foi verificado que

$$r + (p-r) = p,$$

propriedade característica das raízes quadradas modulares em corpos finitos.

5.4. Teste de Falha

Também foi considerado um caso sem solução:

$$x^2 \equiv 3 \pmod{41}.$$

Nesse exemplo, o Critério de Euler indicou que 3 não é resíduo quadrático módulo 41, fazendo com que o algoritmo interrompesse sua execução e reportasse adequadamente a inexistência de solução.

5.5. Testes Aleatórios

Para complementar a validação, foram selecionados aleatoriamente cinco resíduos quadráticos módulo um primo gerado durante a execução do programa. Para cada valor, foi calculada uma raiz quadrada modular e verificado que

$$r^2 \equiv a \pmod{p}.$$

Os resultados obtidos confirmaram a corretude da implementação.

Referências

- [1] BEZERRA,Luan. Enunciado do trabalho final tópicos 4,7,9. Disponível em https://virtual.ufmg.br/20261/pluginfile.php/492595/mod_resource/content/1/Trabalho_algebra_A.pdf Acesso em 5 jun. 2026.
- [2] BEZERRA,Luan. Notas de Aula.Disponível em https://virtual.ufmg.br/20261/pluginfile.php/415461/mod_resource/content/3/Notas_de_Aula_Algebra_Apdf.pdf Acesso em 5. jun 2026
- [3] CHAVES,Daniel. Trabalho de conclusão de curso:Algoritmo para o teste de Primalidade de Miller-Rabin. Disponível em: https://www.researchgate.net/publication/335612186_Algoritmo_para_o_teste_de_Primalidade_de_Miller-Rabin#pf2d Acesso em: 6 jun. 2026
- [4] BAPANAPALLY Jagadish. Formalizations of the Tonelli-Shanks Algorithm in ACL2, Integration by Substitution and the Banach-Tarski Theorem ACL2(r). Disponível em: <https://www.proquest.com/openview/1c86fe582160dae130ec4cd4129de8f8/1?pq-origsite=gscholar&cbl=18750&diss=y>.Acesso em: 6 jun. 2026